

机器学习模型生产环境部署手册机器学习模型生产环境部署手册

文档类型：操作手册 适用角色：ML 工程师 / MLOps 工程师 适用框架：PyTorch 2.x / TensorFlow 2.x 最后更新：2026-06-25**文档类型**：操作手册
适用角色：ML 工程师 / MLOps 工程师
适用框架：PyTorch 2.x / TensorFlow 2.x
最后更新：2026-06-25

文档类型：操作手册 适用角色：ML 工程师 / MLOps 工程师 适用框架：PyTorch 2.x / TensorFlow 2.x 最后更新：2026-06-25**文档类型**：操作手册
适用角色：ML 工程师 / MLOps 工程师
适用框架：PyTorch 2.x / TensorFlow 2.x
最后更新：2026-06-25

1. 部署概述1. 部署概述

1.1 部署模式对比1.1 部署模式对比

部署模式部署模式	延迟延迟	吞吐量吞吐量	成本成本	适用场景适用场景
在线推理（REST API）	在 < 100ms< 100ms	中中	中中	实时推荐、风控实时推荐、风控
批量推理（Batch）	批量分钟级分钟级	极高极高	低低	离线报表、ETL离线报表、ETL
边缘部署边缘部署	< 10ms< 10ms	低低	高高	IoT、移动端IoT、移动端
流式推理流式推理	< 500ms< 500ms	高高	中中	实时日志分析实时日志分析
嵌入式模型（ONNX/Ter	< 5ms< 5ms	极高极高	低低	高吞吐 API 服务高吞吐 API 服务

1.2 MLOps 成熟度模型1.2 MLOps 成熟度模型

级别级别	特征特征	关键能力关键能力
L0 手动**L0** 手动	手动训练 + 手动部署手动训练 + 手动部署	Jupyter NotebookJupyter Notebook
L1 自动化训练**L1** 自动化训练	CI/CD 训练流水线CI/CD 训练流水线	ML PipelineML Pipeline
L2 持续交付**L2** 持续交付	自动部署 + 模型注册自动部署 + 模型注册	Model RegistryModel Registry
L3 持续监控**L3** 持续监控	自动回滚 + 漂移检测自动回滚 + 漂移检测	Model MonitoringModel Monitoring

2. 模型准备2. 模型准备

2.1 模型导出格式

```
import torch
import torch.onnx

# 1. PyTorch TorchScript
class SentimentModel(torch.nn.Module):
    def __init__(self, vocab_size: int, embed_dim: int, num_classes: int):
        super().__init__()
        self.embedding = torch.nn.Embedding(vocab_size, embed_dim)
        self.lstm = torch.nn.LSTM(embed_dim, 128, bidirectional=True, batch_first=True)
        self.classifier = torch.nn.Linear(256, num_classes)

    def forward(self, input_ids: torch.Tensor, attention_mask: torch.Tensor):
        x = self.embedding(input_ids)
        x, _ = self.lstm(x)
        x = x[:, -1, :] #
        return self.classifier(x)

# TorchScript
model = SentimentModel(vocab_size=30000, embed_dim=256, num_classes=3)
model.load_state_dict(torch.load("model_checkpoint.pt"))
model.eval()

scripted_model = torch.jit.script(model)
scripted_model.save("sentiment_model.pt")

# 2. ONNX
dummy_input_ids = torch.randint(0, 30000, (1, 128))
dummy_attention_mask = torch.ones(1, 128)

torch.onnx.export(
    model,
    (dummy_input_ids, dummy_attention_mask),
    "sentiment_model.onnx",
    input_names=["input_ids", "attention_mask"],
    output_names=["logits"],
    dynamic_axes={
        "input_ids": {0: "batch_size", 1: "sequence_length"},
        "attention_mask": {0: "batch_size", 1: "sequence_length"},
        "logits": {0: "batch_size"},
    },
    opset_version=17,
)
```

2.2 模型量化

量化方式	精度损失	推理加速	模型压缩比	实现难度
FP32 → FP16	极低	1.5-2x	2x2x	低
动态量化	低	1.3-1.5x	2x2x	低
静态量化 (int8)	中	2-3x	4x4x	中
QAT (量化感知训练)	低	2-3x	4x4x	高

```
# PyTorch
import torch.quantization

quantized_model = torch.quantization.quantize_dynamic(
    model,
    {torch.nn.Linear, torch.nn.LSTM},
    dtype=torch.qint8
)
torch.save(quantized_model.state_dict(), "model_quantized.pt")
```

3. 服务化部署3. 服务化部署

3.1 FastAPI 推理服务3.1 FastAPI 推理服务

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
import torch
import time
from typing import List

app = FastAPI(title="Sentiment Analysis API", version="2.1.0")

#
class ModelManager:
    def __init__(self):
        self.model = torch.jit.load("sentiment_model.pt")
        self.model.eval()
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.model.to(self.device)
        self.labels = ["negative", "neutral", "positive"]

    @torch.no_grad()
    def predict(self, input_ids, attention_mask):
        input_ids = input_ids.to(self.device)
        attention_mask = attention_mask.to(self.device)
        start = time.perf_counter()
        logits = self.model(input_ids, attention_mask)
        inference_time = (time.perf_counter() - start) * 1000
        probs = torch.softmax(logits, dim=-1)
        pred = torch.argmax(probs, dim=-1)
        return {
            "label": self.labels[pred.item()],
            "confidence": probs.max().item(),
            "scores": {self.labels[i]: probs[0][i].item() for i in range(3)},
            "inference_ms": round(inference_time, 2),
        }

model_manager = ModelManager()

class PredictRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=500)
    return_scores: bool = True

class PredictResponse(BaseModel):
    label: str
    confidence: float
    scores: dict | None
    inference_ms: float
    request_id: str

@app.post("/v2/predict", response_model=PredictResponse)
async def predict(request: PredictRequest):
    # Tokenize transformers tokenizer
    from transformers import AutoTokenizer
    tokenizer = AutoTokenizer.from_pretrained("./tokenizer")
    tokens = tokenizer(
        request.text,
        max_length=128,
        padding="max_length",
        truncation=True,
        return_tensors="pt",
    )

    result = model_manager.predict(

```


3.2 Docker 化部署3.2 Docker 化部署

```
FROM pytorch/pytorch:2.2.0-cuda12.1-cudnn8-runtime

WORKDIR /app

#
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

#
COPY sentiment_model.pt .
COPY model_manager.py .
COPY main.py .

# root
RUN useradd -m -u 1000 mluser && chown -R mluser:mluser /app
USER mluser

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=3s \
  CMD curl -f http://localhost:8000/health || exit 1

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "4"]
```

4. 模型注册与版本管理4. 模型注册与版本管理

4.1 MLflow Model Registry4.1 MLflow Model Registry

```
import mlflow

#
mlflow.set_tracking_uri("http://mlflow.company.com:5000")

with mlflow.start_run(run_name="sentiment-v3.2"):
    mlflow.pytorch.log_model(
        model,
        "sentiment_model",
        registered_model_name="sentiment_analyzer",
    )

    mlflow.log_metrics({
        "accuracy": 0.943,
        "f1_score": 0.912,
        "latency_p99_ms": 45.2,
    })

#
client = mlflow.tracking.MlflowClient()
client.transition_model_version_stage(
    name="sentiment_analyzer",
    version=5,
    stage="Production",
    archive_existing_versions=True,
)
```

4.2 模型版本对比4.2 模型版本对比

版本版本	阶段阶段	准确率准确率	P99 延迟P99 延迟	发布日期发布日	状态状态
v3.2v3.2	ProductionProduct	94.3%94.3%	45ms45ms	2026-07-012026-07-01	当前当前
v3.1v3.1	ArchivedArchived	93.8%93.8%	48ms48ms	2026-06-152026-06-15	已归档已归档
v3.0v3.0	ArchivedArchived	93.2%93.2%	52ms52ms	2026-05-202026-05-20	已归档已归档
v4.0-beta v4.0-beta	StagingStaging	95.1%95.1%	38ms38ms	2026-07-102026-07-10	测试中测试中

5. 模型监控与漂移检测5. 模型监控与漂移检测

5.1 核心监控指标5.1 核心监控指标

```

class ModelMonitor:
    """

    def __init__(self):
        self.metrics = {
            "prediction_count": 0,
            "error_count": 0,
            "latency_sum_ms": 0,
            "label_distribution": {"negative": 0, "neutral": 0, "positive": 0},
            "confidence_distribution": [],
        }

    def record_prediction(self, result: dict, latency_ms: float):
        self.metrics["prediction_count"] += 1
        self.metrics["latency_sum_ms"] += latency_ms
        self.metrics["label_distribution"][result["label"]] += 1
        self.metrics["confidence_distribution"].append(result["confidence"])

    def detect_drift(self, reference_distribution: dict) -> dict:
        """ KS """
        from scipy.stats import ks_2samp

        current = self.metrics["label_distribution"]
        total_current = sum(current.values())
        total_ref = sum(reference_distribution.values())

        current_probs = [v / total_current for v in current.values()]
        ref_probs = [v / total_ref for v in reference_distribution.values()]

        ks_stat, p_value = ks_2samp(current_probs, ref_probs)
        return {
            "drift_detected": p_value < 0.05,
            "ks_statistic": ks_stat,
            "p_value": p_value,
            "severity": "high" if p_value < 0.01 else "medium" if p_value < 0.05 else "low",
        }

```

5.2 自动回滚策略

触发条件	阈值	动作	通知
准确率下降	< 90% (持续 10 分钟)	< 90% 自动回滚到上一版本	邮件 + 即时通讯
P99 延迟	> 200ms (持续 5 分钟)	> 200 自动回滚	即时通讯
错误率	> 1% (持续 3 分钟)	> 1% (持续 3 分钟) 自动回滚	电话 + 即时通讯
数据漂移	$p < 0.01$	发送告警，人工决策	邮件

6. A/B 测试与灰度发布

6.1 流量分割配置

```
# Istio VirtualService
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: sentiment-api
spec:
  hosts:
  - sentiment-api.production.svc.cluster.local
  http:
  - match:
    - headers:
        x-model-version:
          exact: "beta"
    route:
    - destination:
        host: sentiment-api
        subset: v4-beta
      weight: 100
    - route:
        - destination:
            host: sentiment-api
            subset: v3-stable
          weight: 90
        - destination:
            host: sentiment-api
            subset: v4-beta
          weight: 10
```

7. 常见部署问题7. 常见部署问题

问题问题	原因原因	解决方案解决方案
推理结果不一致推理结果不一致	预处理不一致预处理不一致	统一 tokenizer 和预处理 pipeline统一 tokeni
GPU OOMGPU OOM	Batch size 过大Batch size 过大	调整 batch size 或使用梯度累积调整 batch size
模型加载慢模型加载慢	模型体积大模型体积大	使用 TorchScript/ONNX 序列化使用 TorchScri
冷启动延迟高冷启动延迟高	模型加载在线程中模型加载在线程中	使用 warmup 请求预热模型使用 warmup 请求预热模型

8. 总结8. 总结

ML 模型生产部署的关键要点：ML 模型生产部署的关键要点：

- 模型优化：量化、剪枝、TorchScript/ONNX
导出是降低推理延迟的三板斧**模型优化**：量化、剪枝、TorchScript/ONNX
导出是降低推理延迟的三板斧
- 服务化：FastAPI + Docker + K8s 是当前最成熟的在线推理技术栈**服务化**：FastAPI +
Docker + K8s 是当前最成熟的在线推理技术栈

3. 版本管理：MLflow Model Registry 实现模型的完整生命周期管理**版本管理**：MLflow Model Registry 实现模型的完整生命周期管理
4. 监控必备：延迟、错误率、数据漂移三者缺一不可**监控必备**：延迟、错误率、数据漂移三者缺一不可
5. 灰度策略：从 5% → 25% → 100% 的渐进式发布是安全生产的基础**灰度策略**：从 5% → 25% → 100% 的渐进式发布是安全生产的基础

本手册由 MLOps 团队维护，每季度融合新的最佳实践。